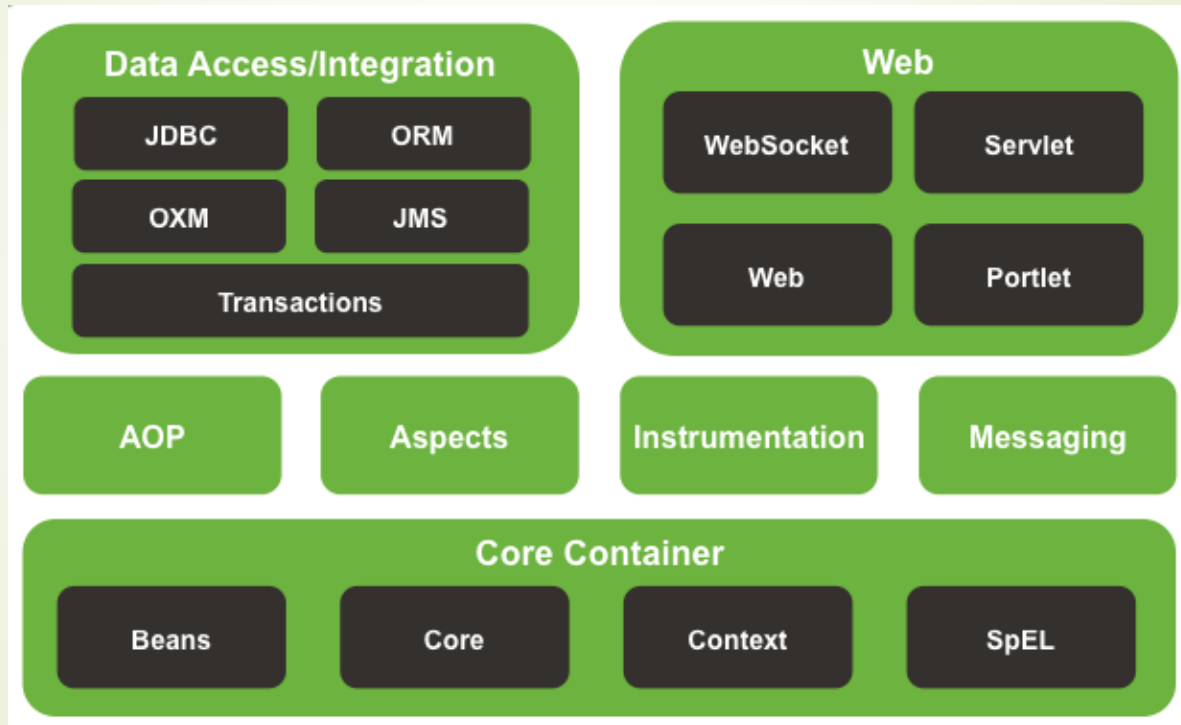


Spring Framework



Módulos

Spring se compone de siete módulos, siendo el núcleo la inversión del control.



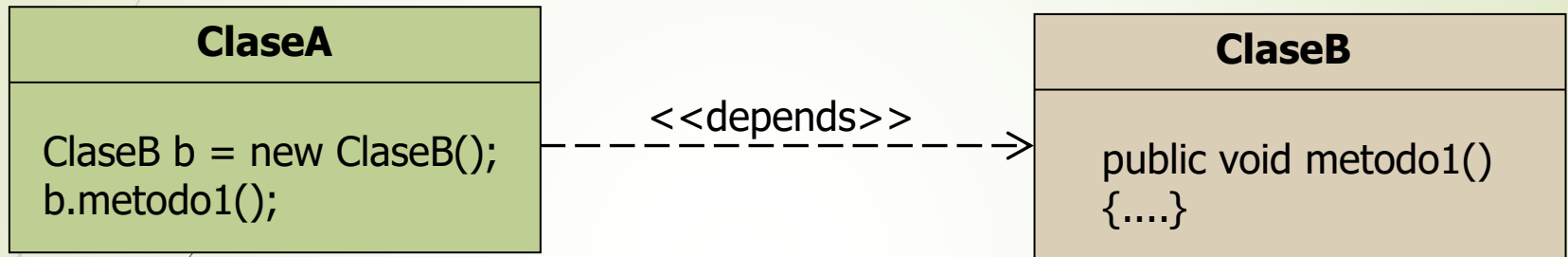
Tipos de Aplicaciones

Spring permite la creación de todo tipo de aplicaciones: aplicaciones web, aplicaciones multicapa, clientes ricos.

- **No es invasivo:** permite su uso conjunto a otros frameworks (p.e.: Struts o Hibernate).
- **No es todo o nada:** no obliga a la implementación de toda la aplicación empleando Spring.
- **Aplicaciones empresariales "ligeras" y basadas en Java EE:** no requieren de un servidor de aplicaciones, solamente una máquina virtual. Pero también se pueden integrar con servidores JEE para obtener las ventajas que estos ofrecen (Seguridad frente a fallos, escalabilidad, etc).
- **Sistema declarativo de manejo de transacciones:** basado en AOP puede manejar de forma declarativa las de un único recurso transaccional (p.e.: un DataSource). Si se desean realizar transacciones globales (que agrupen varios recursos transaccionales) se deberá usar una transacción de un servidor JEE mediante JTA.

Dependencias

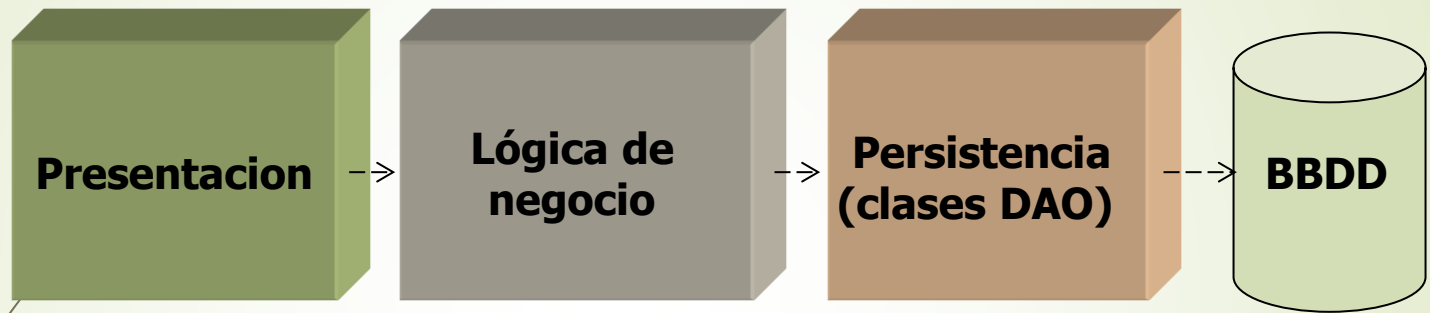
Se dice que una clase depende de otra cuando la primera tiene variables cuyo tipo es de la segunda y cuando la primera crea objetos de la segunda.



- 1.- Difícil mantenimiento y rediseño: En tiempo de diseño, cualquier cambio realizado en una clase, puede afectar a las clases que dependen de ella y por lo tanto que sea necesario modificarlas también.
- 2.- Menor estabilidad: En tiempo de ejecución, si una parte de la aplicación tiene un error o deja de funcionar, si el sistema tiene muchas dependencias, se verá mayormente afectado.

Reducir el acoplamiento

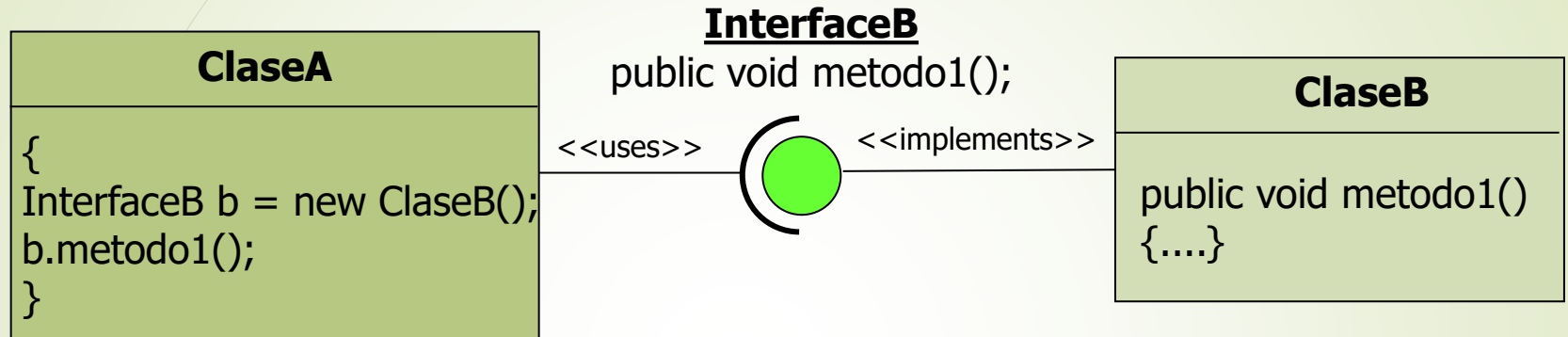
Capas



- Cada capa solamente se comunica con las capas contiguas y por lo tanto solamente depende de ellas.
- Ej: No hay dependencia entre la capa de Presentación y la de Persistencia

Reducir el acoplamiento

Interfaces



Con el uso de interfaces, las variables empleadas en una parte de la aplicación no quedan ligadas a una clase sino a una interface.

Reducir el acoplamiento

Inyección de dependencias (DI)

ClaseA
<pre>{ InterfaceB b = new ClaseB(); b.metodo1(); }</pre>

DI por parámetro

ClaseA
<pre>InterfaceB b; public void setB(InterfaceB b){ this.b=b; } ... b.metodo1();</pre>

DI por constructor

ClaseA
<pre>InterfaceB b; public ClaseA (InterfaceB b){ this.b=b; } ... b.metodo1();</pre>

Reducir el acoplamiento DI con Spring

Primer paso hacia la Ioc

Obtención de objetos mediante una clave a través de un contexto de Spring.

Ioc pura

Asignación del objeto por el contexto mediante un método "set" de la claseA

ClaseA

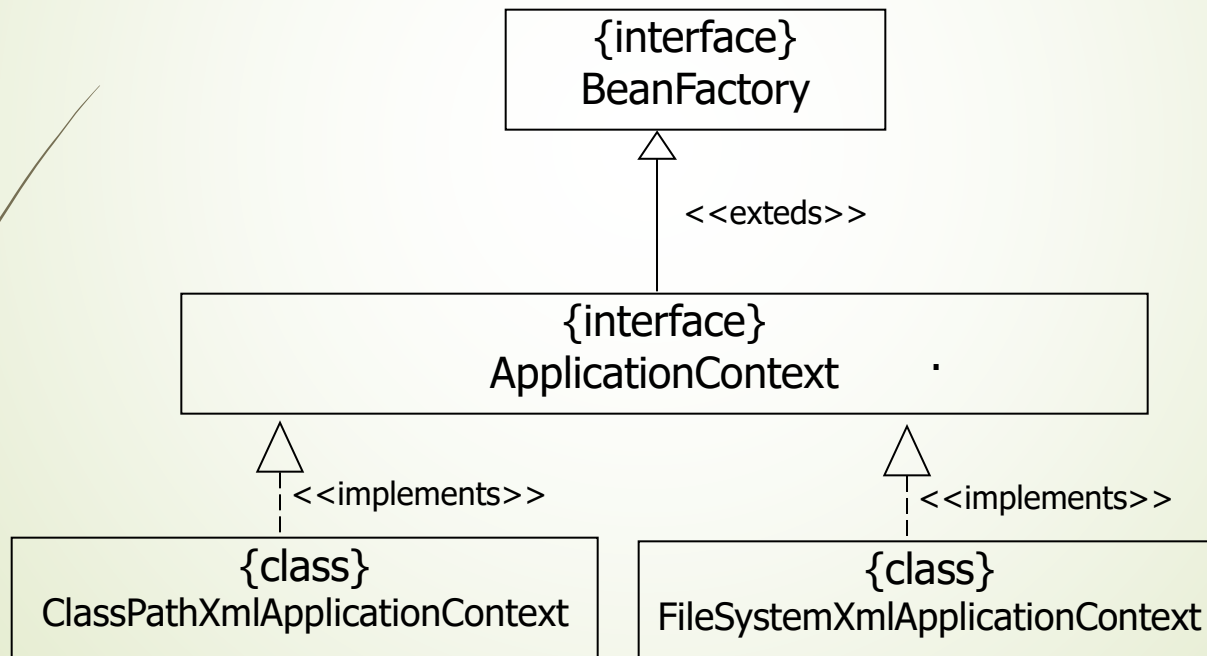
```
{  
  InterfaceB b = ctx.getBean("claveObjetoB");  
  b.metodo1();  
}
```

ClaseA

```
InterfaceB b;  
  
public void setB(InterfaceB b){  
    this.b=b;  
}  
  
...  
b.metodo1();
```


Factoría de Beans Contexto

Contienen la funcionalidad para la obtención de beans y además, permiten configurar otros servicios del framework. (internacionalización, recursos web, AOP entre otros).



Obtención del Contexto

Una factoría de beans se puede obtener a partir de uno o varios documentos xml, a partir de un archivo properties o por una clase java programada adhoc (JavaConfig)

```
// Creación de un ApplicationContext a partir de un xml que está en el classpath.  
BeanFactory ctx = new ClassPathXmlApplicationContext("applicationContext.xml");
```

```
// Creación de un ApplicationContext a partir de un xml que está en el un directorio.  
BeanFactory ctx = new FileSystemXmlApplicationContext ("applicationContext.xml");
```

Los contextos se pueden parametrizar con archivos de properties a cuyos valores se accede desde el xml con la forma `${}`. Para esto, se debe indicar un bean que referencie al archivo de properties:

```
<context:property-placeholder location="classpath:bbdd.properties" />
```

Contextos Jerárquicos

Al crear un contexto, se le puede pasar como segundo parámetro una referencia a otro contexto que será su contexto "padre". De forma que si se realiza una búsqueda de un bean en el contexto "hijo" y no se encuentra, se buscará en el contexto padre.

```
ApplicationContext ctxPadre = new ClassPathXmlApplicationContext("ctx1.xml");
```

```
ApplicationContext ctxHijo =  
    new ClassPathXmlApplicationContext(new String []{"ctx2.xml"}, ctxPadre);
```

Spring

Librerías necesarias

Dependencias de Spring en maven.

Core

```
<dependency>  
    <groupId>org.springframework</groupId>  
    <artifactId>spring-context</artifactId>  
    <version>version</version>  
</dependency>
```

Spring

Librerías necesarias

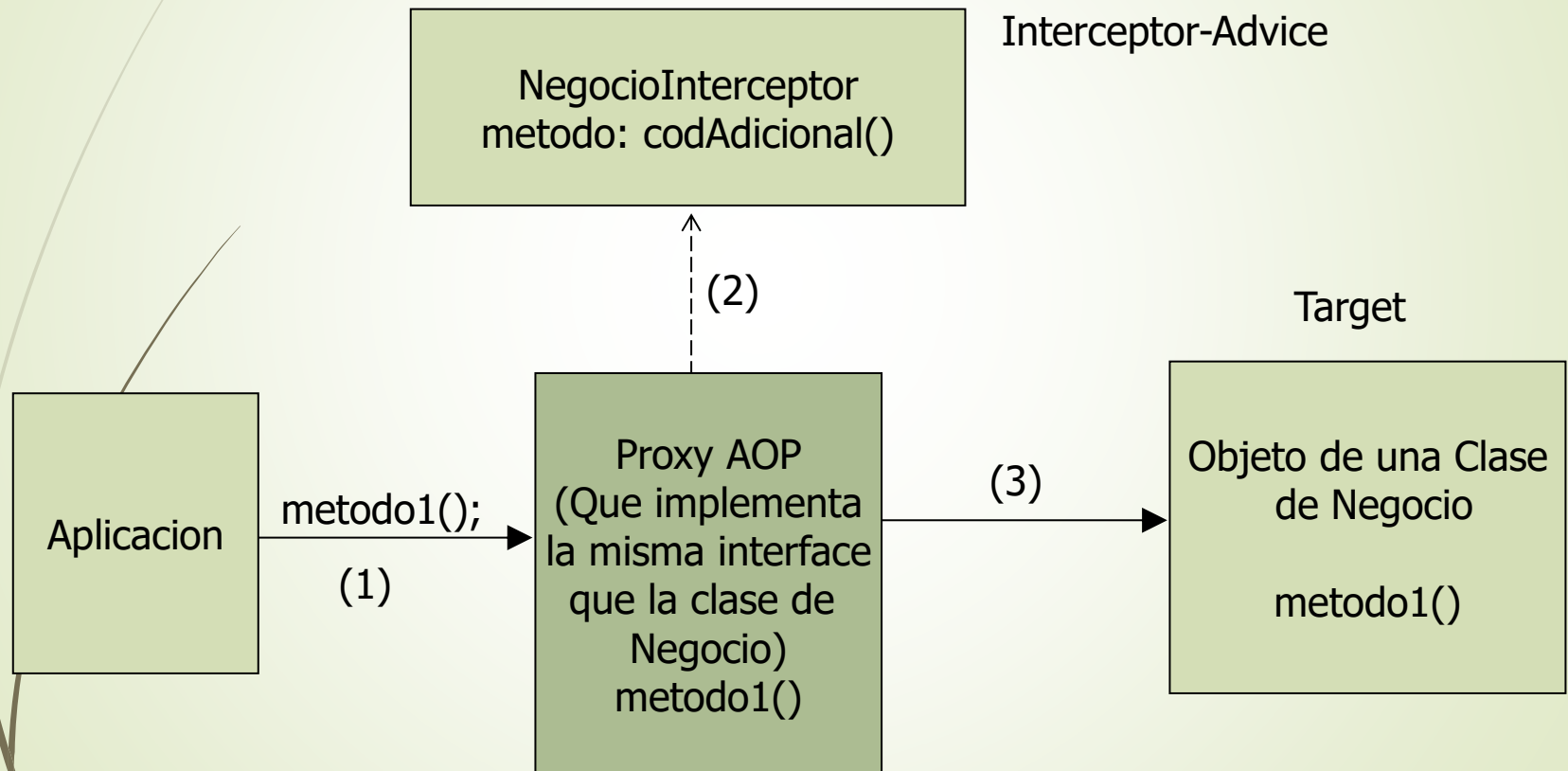
Dependencias de Spring Aspects en maven.

AOP

```
<dependency>  
  <groupId>org.springframework</groupId>  
  <artifactId>spring-aspects</artifactId>  
  <version>version</version>  
</dependency>
```

Spring AOP

Cuando la aplicación le solicita al contexto un bean de negocio, el contexto le suministra una referencia al Proxy que implementa la misma interface que el Objeto de Negocio. Cuando la aplicación ejecuta el método de negocio, el proxy llama primero al invoke del interceptor.



Spring AOP

Terminología

Cross-cutting concerns (problema transversal): Determinadas clases de una aplicación, comparten requerimientos secundarios comunes. Por ejemplo, añadir logging a las clases de acceso a los datos y a las clases de negocio cada vez que se ejecuta uno de sus métodos. Aunque la funcionalidad principal de cada clase sea muy diferente, el código necesario para realizar la segunda funcionalidad es casi el mismo.

Advice (Consejo): Este es el código adicional que se quiere aplicar al modelo existente. En el ejemplo, es el código de logging que se quiere ejecutar cada vez que se entre en un método de las clases de datos o negocio.

Point-cut (punto de ruptura o punto de corte): Conjunto de puntos de ejecución de la aplicación (que cumplen un patrón o plantilla) donde se desea aplicar el código adicional de un **Advice**. En el ejemplo, se tendrá un punto de corte en la entrada de los métodos de las clases de acceso a datos y negocio. Cada uno de estos puntos de ejecución se llama **Joinpoint**.

Aspect (Aspecto): La combinación de un punto de corte y un consejo es un aspecto. En el ejemplo, será la aplicación del consejo (advice) de logging a los puntos de corte (point-cut) de los métodos de negocio y acceso a datos.

Spring AOP

Espacio de nombres aop:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:aop="http://www.springframework.org/schema/aop"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
    http://www.springframework.org/schema/aop
    http://www.springframework.org/schema/aop/spring-aop-3.0.xsd">

</beans>
```

Tipos de Interceptores

Se pueden declarar beans interceptores de 5 tipos, dependiendo de cuándo se ejecutarán.

Antes de invocar al método	<code><aop:before method=""/></code>
Alrededor (Encapsula al método)	<code><aop:around method=""/></code>
Después de invocar al método	<code><aop:after method=""/></code>
Después de finalizar con éxito	<code><aop:after-returning method=""/></code>
Después de lanzar una excepción	<code><aop:after-throwing method=""/></code>

Tipos de Interceptores

```
<aop:config>

  <aop:aspect ref="beanBeforeAspect">
    <aop:before method="antes"
      pointcut="execution(* agenda.AgendaVectorDao.buscarPorNombre(..) and args(nom))"/>
  </aop:aspect>

  <aop:aspect ref="beanAfterAspect">
    <aop:after-returning returning="v" method="despues"
      pointcut="execution(* agenda.AgendaVectorDao.buscarPorNombre(..))"/>
  </aop:aspect>

  <aop:aspect ref="beanAroundAspect" order="1">
    <aop:around method="alrededor"
      pointcut="execution(* agenda.AgendaVectorDao.buscarPorNombre(..) and args(nom))"/>
  </aop:aspect>

</aop:config>
```

Pointcut

AspectJ pointcut designators (PCD)

execution: para referenciar join points de ejecución del método

within: para referenciar join points dentro de ciertos tipos (clase o paquete.*)

target: join points de un tipo determinado (Clase)

args: join points donde los argumentos son instancias de los tipos dados

@target: join points donde la clase del objeto en ejecución tiene una anotación del tipo dado

@args: join points donde el tipo en tiempo de ejecución de los argumentos reales pasados tiene anotaciones de los tipos dados

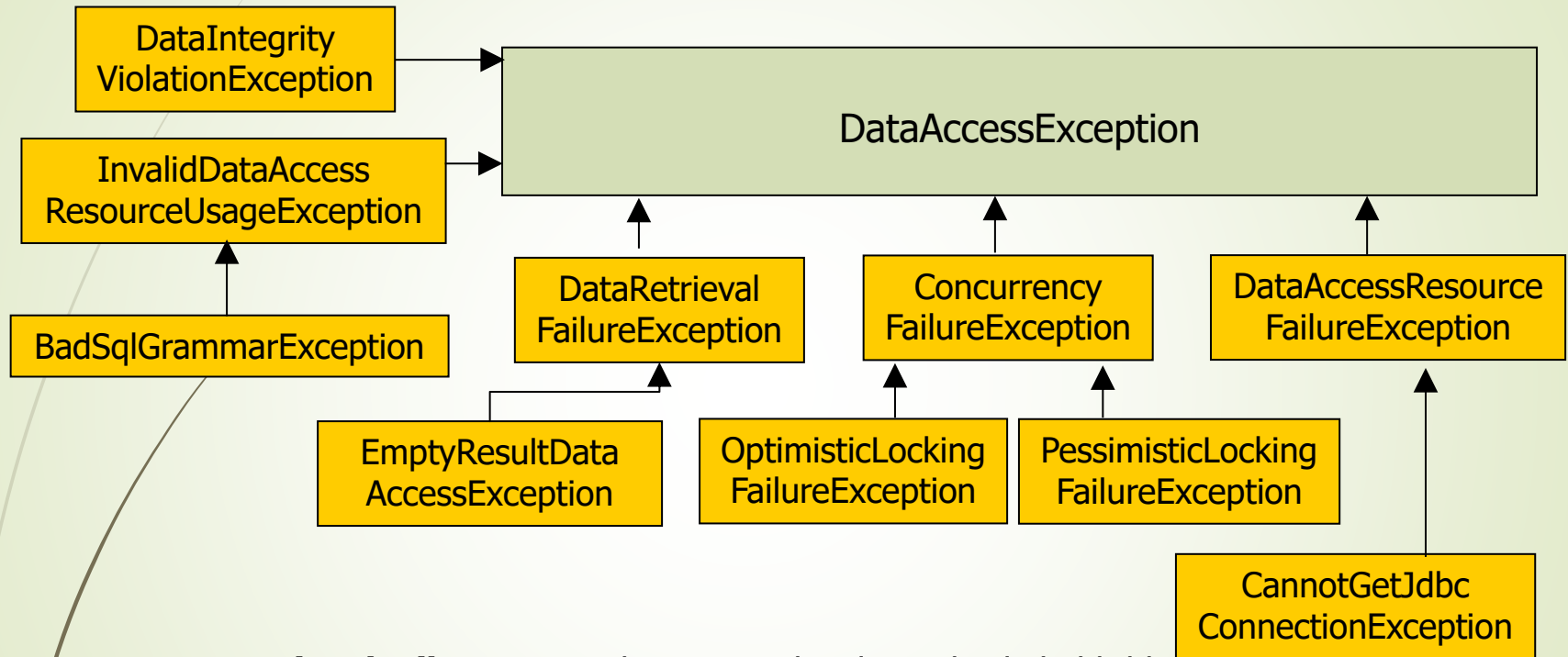
@annotation: join points donde el método tiene la anotación dada

Spring DAO

Permite la implementación del Patrón DAO a partir de Clases específicas para distintas tecnologías de persistencia

- JDBC
- Hibernate
- JDO
- iBatis
- JPA

Spring DAO Excepciones



DataRetrievalFailure: Error al recuperar los datos desde la bbdd.

DataAccessResourceFailure: Error al conectar con la bbdd.

DataIntegrityViolation: Cuando se ha violado alguna regla de integridad. Ej. clave primaria duplicada.

Spring - acceso a datos con JPA

Los DAO para JPA se implementan directamente con el objeto EntityManager de JPA. Sin embargo para el manejo de transacciones de forma declarativa, ese objeto será inyectado por el contexto y las excepciones de JPA podrán ser traducidas a las de Spring.

Los daos JPA se pueden declarar como beans mediante anotaciones @Repository en lugar de declararlos en el xml. Para ello en el contexto debe estar presente la etiqueta:

```
<context:component-scan base-package="persistencia,negocio,presentacion" />
```

La anotación @Repository además es la encargada de que funcione la translación de excepciones, de modo que cada método del dao que lance hacia fuera una excepción de JPA, esta será traducida a la jerarquía de excepciones de Spring.

```
@Repository //Para que funcione la translación de excepciones de JPA a Spring  
public class BancoDao implements BancoDaoInterface
```

En el contexto de Spring deberá estar declarado un bean encargado de la traducción de excepciones:

```
<bean class="org.springframework.dao.annotation.PersistenceExceptionTranslationPostProcessor" />
```


Transacciones

Características (ACID)

ÁTÓMICO

Las transacciones están compuestas de una o más actividades agrupadas como una sola unidad de trabajo.

CONSISTENTE

Finalizada una transacción, de forma correcta o no, el sistema debe quedar en un estado consistente.

AISLADO

Las transacciones deben soportar acceso a múltiples usuarios, de forma que las transacciones deben aislarse una de otras y evitar el acceso concurrente.

DURABLE

Una vez finalizada una transacción, el resultado debe persistir y tolerar cualquier tipo de fallo del sistema.

Transacciones

¿ DÓNDE DEBEN CONTROLARSE LAS
TRANSACCIONES?

¿ EN LOS DAOS ?

SE DEBEN CONTROLAR A NIVEL DE SERVICIO
LA LÓGICA DE NEGOCIO DEFINE LAS
TRANSACCIONES

Spring: Gestión declarativa de Transacciones

Permite especificar declarativamente qué métodos de negocio constituyen una transacción. Mediante proxies de programación orientada a aspectos se maneja el datasource para deshacer la transacción si se detecta alguna excepción.

- No necesita un servidor de aplicaciones para ejecutarse.
- No es dependiente de la tecnología de acceso a datos(Jdbc, Hibernate, etc.).
- Se puede deshacer una transacción de forma programática mediante el método *setRollBackOnly()* de la clase *TransactionStatus* aunque esto está desaconsejado ya que se recomienda la forma declarativa.
- Se pueden declarar reglas que indiquen que excepciones provocan un rollback.

Spring - Gestión declarativa de Transacciones

```
@Transactional  
public void metodoTransaccional() {  
    ...  
}
```

- Una tx comienza justo antes de la primer línea del método
- Finaliza justo después de la última línea
- Si llama a un método cualquiera que no está anotado con @Transactional, este entra dentro de la transacción

Spring - Gestión declarativa de Transacciones

Contexto

En el contexto se debe declarar el TransactionManager , el EntityManagerFactory e indicar la etiqueta tx:annotation-driven. El archivo de persistencia podra no llamarse persistence.xml.

```
<tx:annotation-driven transaction-manager="transactionManager"/>

<bean id="transactionManager" class="org.springframework.orm.jpa.JpaTransactionManager">
    <property name="entityManagerFactory" ref="emf" />
</bean>

<bean id="emf" class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
    <property name="persistenceXmlLocation" value="classpath:META-INF/persistence_spr.xml" />
    <property name="persistenceUnitName" value="bancoJpa" />
    <property name="dataSource" ref="dataSource" />
    <property name="jpaVendorAdapter">
        <bean class="org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter"/>
    </property>
</bean>
```

Spring - Gestión declarativa de Transacciones

Archivo de persistencia (por ejemplo "persistence_spr.xml")

En el archivo de persistencia no se dan los datos de conexión a la bbdd (ya lo hace el dataSource). Normalmente es suficiente con declarar los Entities a emplear.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1" xmlns="http://xmlns.jcp.org/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
    http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd">
  <persistence-unit name="bancoJpa" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.ejb.HibernatePersistence</provider>
    <class>entities.ClienteEntity</class>
    <class>entities.ProvinciaEntity</class>
    <class>entities.TarjetaEntity</class>
    <class>entities.CuentaEntity</class>
    <properties>
    </properties>
  </persistence-unit>
</persistence>
```

Spring - Gestión declarativa de Transacciones

Código java

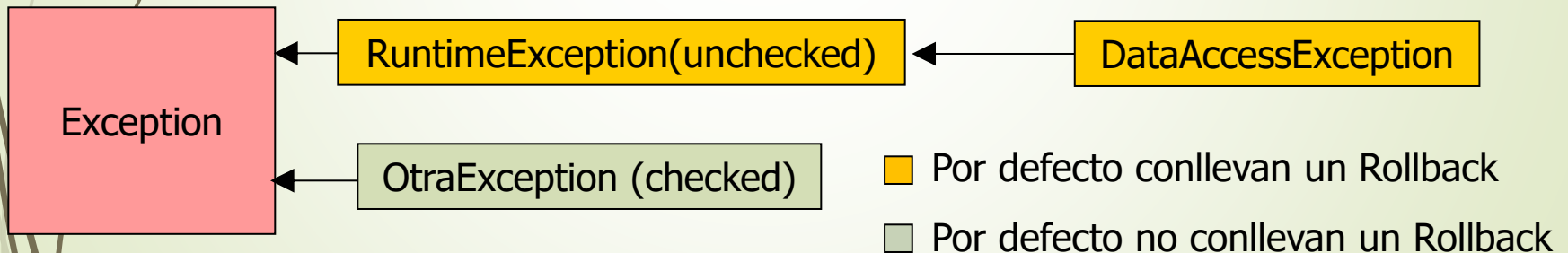
Para indicar que un método de negocio es transaccional se antepone la anotación @Transactional

```
@Transactional(propagation=Propagation.REQUIRED,  
                rollbackFor=ClienteSinSaldoException.class)  
public void transferenciaSiHaySaldo(int dni1, int dni2, double importe)  
    throws ClienteSinSaldoException {  
    // ...  
}
```


Transacciones Spring Rollback

En la anotación `@Transactional` se puede especificar qué excepciones producen rollback y cuáles no.

```
@Transactional (rollbackFor={ClienteYaExisteException.class},  
                noRollbackFor={SaldoNegativoException.class})
```



Propagación de Transacciones

- REQUIRED:** El método siempre es llamado con una transacción, bien una existente en el contexto desde el que es llamado, o en caso contrario, se le asociará una nueva transacción.
- REQUIRES_NEW:** El método se ejecuta en una nueva transacción aunque sea llamado desde un ambito transaccional.
- NOT_SUPPORTED:** El método no es transaccional por si mismo ni se incluirá dentro de ninguna transacción. Si el llamado y la tx existe, la pone en suspenso y se retoma al finalizar.
- SUPPORTS:** Si el método es llamado desde un contexto transaccional, se incluirá en la transacción. En caso contrario será un método no transaccional.
- MANDATORY:** Solamente puede ser llamado desde un contexto transaccional, de lo contrario produce una excepción.
- **NEVER:** No puede ser llamado desde un ambito transaccional. Si se hace, lanza una excepción.

Propagación de Transacciones

Atributo	existe transacción previa	Efecto
REQUIRED	No	El contenedor crea una nueva transacción
	Si	El método se une a la transacción de quien le llama
REQUIRES_NEW	No	El contenedor crea una nueva transacción
	Si	Se crea una nueva transacción y la del llamador se pausa.
SUPPORTS	No	No se usa transacción
	Si	Se suma a la transacción de quien le llama.
MANDATORY	No	Se lanza una Exception
	Si	Se suma a la transacción de quien le llama.
NOT_SUPPORTED	No	No se usa transacción
	Si	No se usa transacción y la transacción de quien le llama se pausa.
NEVER	No	No se usa transacción.
	Si	Lanza una Exception

Spring Data - JPA

SpringData genera de forma automática los Daos de las clases persistentes.

1. Incorporar el espacio de nombres jpa al applicationContext.xml
2. Incorporar la etiqueta jpa:repositories
`<jpa:repositories base-package="lista de paquetes"/>`
 - Buscará todas las interfaces que hereden de `JpaRepository<T , Object>`
 - Instanciará automáticamente los daos, que implementarán la interfaz creada

```
public interface ClienteRespository extends JpaRepository<Cliente, Integer>{}
```

Instanciará un obj que implemente esta interfaz, con 20 métodos:
`save(...)`, `findById(...)`, `findAll(...)`, `delete(...)`, `deleteAll(...)`, `exist(...)`, `flush()`, etc.

*En lugar de `JpaRepository`, podemos exponer `CrudRepository`

Spring Data - JPA

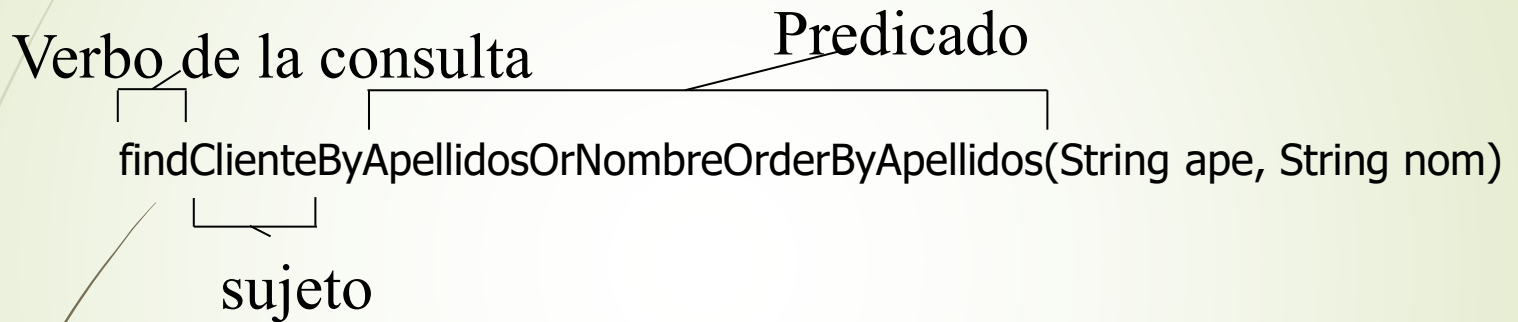
Se puede extender la funcionalidad de JpaRepository

```
public interface ClienteRespository extends JpaRepository<Cliente, Integer>{  
  
    // Mediante el nombre SpringData construye la consulta  
    tipoRetorno verboSujetoByPredicado(parametros);  
  
    @Query(consulta JPA o consulta SQL nativa)  
    declaración normal de método  
  
}
```

Spring Data - JPA

Extensión de la funcionalidad de JpaRepository

Opción 1: Crea las consultas analizando el nombre del método



verbo: get read find count (los 3 primeros son sinónimos)

Sujeto: se ignora, excepto si comienza con Distinct

Predicado:

atributoOperador + And/Or + ... + [IgnoringCase] [OrderBy]

```
public List<Cliente> findByNombreOrApellidosAllIgnoringCase(String nom, String ape);
```

Spring Data - JPA

Opción 1: Crea las consultas analizando el nombre del método

Palabra Clave	Ejemplo	Clausula where generada
And	findByLastnameAndFirstname	... where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	... where x.lastname = ?1 or x.firstname = ?2
Is,Equals	findByFirstname,findByFirstnames,findByNameEquals	... where x.firstname = ?1
Between	findByStartDateBetween	... where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	... where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	... where x.age ≤ ?1
GreaterThan	findByAgeGreaterThan	... where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	... where x.age ≥ ?1
After	findByStartDateAfter	... where x.startDate > ?1
Before	findByStartDateBefore	... where x.startDate < ?1
IsNull	findByAgeIsNull	... where x.age is null
IsNotNull,NotNull	findByAge(Is)NotNull	... where x.age not null
Like	findByFirstnameLike	... where x.firstname like ?1

Spring Data - JPA

Opción 1: Crea las consultas analizando el nombre del método

Palabra Clave	Ejemplo	Clausula where generada
NotLike	findByFirstnameNotLike	... where x.firstname not like ?1
StartingWith	findByFirstnameStartingWith	... where x.firstname like ?1 (parameter bound with appended %)
EndingWith	findByFirstnameEndingWith	... where x.firstname like ?1 (parameter bound with prepended %)
Containing	findByFirstnameContaining	... where x.firstname like ?1 (parameter bound wrapped in %)
OrderBy	findByAgeOrderByLastnameDesc	... where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot	... where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	... where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	... where x.age not in ?1
True	findByActiveTrue()	... where x.active = true
False	findByActiveFalse()	... where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	... where UPPER(x.firstname) = UPPER(?1)

Spring Data - JPA

Opción 2: Consulta JPQL y Nativas con la anotación @Query

```
public interface ClienteRepository extends JpaRepository<Cliente, Integer> {

    @Query("select c from Cliente c where c.apellidos like ?1%")
    List<Cliente> findByApellidos(String apellido);

    // Parámetros con nombre
    @Query("select c from Cliente c where c.nombre= :nombre and
           c.apellidos = :apellidos")
    List<Cliente> findByNombreCompleto(@Param("apellidos") String ape,
                                       @Param("nombre") String nom);

    // Query Nativa
    @Query(value = "SELECT * FROM CLIENTES WHERE EMAIL = ?1",
           nativeQuery = true)
    Cliente findByCorreo(String correo);
}
```

Spring Data - JPA

Opción 3: Crear una clase con métodos implementados. Se “heredarán” en nuestro repositorio

1. Crear una interfaz con la declaración de los métodos, el nombre es irrelevante
2. Crear una clase que implemente esta interfaz con los métodos implementados.
 1. Como atributo, es muy probable que se necesite el EntityManager, irá entonces anotado como `@PersistenceContext`
 2. Podría ser útil tener disponibles los métodos CRUDs para utilizar en el método, para ello incorporamos un atributo de la interfaz del repositorio como `@Autowired`
 3. Su nombre debe ser igual a la interfaz repositorio y un sufijo, por defecto debe ser `Impl`.
 4. Para utilizar otro sufijo, debemos incluir en la configuración:
`repository-impl-postfix="OtroSufijo"` en la etiqueta `<jpa:repositories>`
*`<jpa:repositories base-package="persistencia" />` Busca los finalizados en **Impl***
`<jpa:repositories base-package="persistencia" repository-impl-postfix="PLG"/>`
3. La interfaz repositorio, extenderá de `JpaRepository` y de la interfaz nueva

Spring Data - JPA

Opción 3: Crear una clase con métodos implementados. Se “heredarán” en nuestro repositorio

```
public interface ClienteRepositoryCustomizado {  
    public void miMetodo(Cliente c); }  
  
Public class ClienteRepositoryImpl implements ClienteRepositoryCustomizado {  
    @PersistenceContext  
    private EntityManager em; // no es obligatorio  
    @Autowired  
    private ClienteRepository dao; // no es obligatorio  
  
    public void miMetodo(Cliente c){  
        // Código del método  
    }  
  
    public interface ClienteRepository extends JpaRepository<Cliente, Integer>,  
        ClienteRepositoryCustomizado {  
        ...  
    }  
}
```